



How I build a PoV

Five stages. One written plan does the work.

Here's the whole thing, so you can run it yourself next week.

O que vamos ver

01 O ciclo, de ponta a ponta

02 O plano é o produto

03 O build contra Atlas real

04 O argumento que a PoV faz

05 Onde o conhecimento fica

06 Como rodar na sua próxima PoV

Cinco etapas, um artefato cada

01	Obsidian vault	ENTREGA A decisão e o que recusamos	RESOLVE O contexto do agente zera toda sessão. Um repo nunca conta quais opções recusamos, nem por quê.
02	implementation_plan.md	ENTREGA Claim, ordem de trabalho, demo script	RESOLVE Prompt ad-hoc gera uma PoV diferente cada vez. Este é o artefato que carrega o método.
03	Claude Code + MCP	ENTREGA Código contra o schema real	RESOLVE Sem MCP o agente programa contra um schema que ele imaginou.
04	Atlas real + Playwright	ENTREGA Demo executável e prints do que rodou	RESOLVE Demo em dado falso não responde nenhuma pergunta que o cliente faz.
05	graphify	ENTREGA Grafo do repo, de volta ao vault	RESOLVE O conhecimento morria no laptop de quem construiu.

05 → 01 A PoV número 10 começa com o que as nove primeiras aprenderam.

O que entra no plano

9

planos no portfólio

SEÇÃO

195–254

linhas cada — curto de propósito

254

linhas geraram a PoV MultiAgent inteira

O QUE ELA FIXA

O claim, na primeira linha	o que esta PoV tem que provar — não o stack
Agentes · roteamento · guardrail · cache	quem decide, em que ordem, em qual threshold
Recuperação e busca	índices, campos, pipeline
Frontend	só as telas que o demo script precisa
Portas e comandos	como sobe, como é testado
Ordem de trabalho	sequência de build — e o que não pode ser reordenado
O demo script que eu quero no fim	é isto que significa "pronto": demo executável, não lista de features
Lacunas conhecidas — anotar, não corrigir	mata gold-plating; é o que mantém o plano em ~250 linhas

As duas últimas linhas são o que faz o método funcionar. Tudo acima delas é descrição; essas duas são o que impede um build de derivar.

Com o que o agente trabalha

MongoDB MCP

Cria projeto e cluster, lista coleções, cria índices de Atlas Search e Vector Search, roda agregações, lê o schema real.
Sem ele, o agente programa contra um schema que inventou.

Playwright MCP

Abre a aplicação, percorre o demo script, captura os prints do README.
Teste de aceite e captura de evidência são o mesmo passo — um print só existe se o cenário rodou de verdade.

RESOLVIDO DE ANTEMÃO

PORTS.md — reserve a porta primeiro; nove PoVs dividem um laptop e nenhum launcher mata quem está escutando.
POV_UI_DESIGN_SYSTEM.md — copie os tokens, adapte só o acento do domínio.
Ninguém gasta tempo de PoV escolhendo porta ou cor.

CAMADA DE CLAUDE.MD

O QUE ELA CARREGA

~/claude/ — global

como quero que o agente trabalhe em qualquer lugar

PoVs/ — workspace

convenções comuns às nove, incluindo o registro de portas

<pov>/ — projeto

stack, portas e comandos daquela PoV

O agente não adivinha nossas convenções. Ele as lê.

Cada feature tem que merecer seu lugar

FEATURE	EM MULTIAGENT	POR QUE ESTA
Document model	agent_registry guarda modelo, persona, tools e budget de cada agente	config do agente vira dado — trocar de modelo ao vivo é um update_one, sem redeploy
Atlas Vector Search	catálogo, KB, guardrail semântico, todas as camadas de cache	mesmo cluster do dado operacional; a alternativa é um vector DB separado com ETL e drift
Hybrid search (RRF)	kb_articles via \$unionWith	recall semântico com precisão de termo exato num só pipeline
Cache em 3 níveis	cascata de cache	um round trip decide HIT/MISS antes de qualquer chamada de LLM
Change Streams	agent_handoffs → SSE → timeline na UI	observabilidade ao vivo sem broker; a alternativa é rodar Kafka só para isso
TTL indexes	memória 24h, auditoria 30d, eval_runs 90d	ciclo de vida do dado como política do banco, não cron job
\$jsonSchema	agent_handoffs, agent_traces	o MongoDB rejeita documento malformatado, não só a camada Python

01 VAULT — O PORQUÊ / 05 GRAPHIFY — O QUÊ

Temporal, Kafka/Redis Pub-Sub e vector DB dedicado ficam registrados como recusados, com a razão de cada um. **8.1x** menos tokens por pergunta, **4** bugs achados em 5 PoVs. Os dois alimentam o próximo plano.

Starting your next PoV

- 1 Vault note with the decision and what you rejected
- 2 Write the plan — claim first, then the demo script and known gaps
- 3 Reserve a port, copy the UI tokens
- 4 Run the plan in Claude Code, MCP on real Atlas
- 5 Run the demo script through Playwright, capture screenshots
- 6 `graphify --update`, decision back to the vault

Still rough, so you know going in: long plans cost real tokens · screenshots need recapturing when the UI changes · the graph goes stale if nobody re-indexes · credential hygiene across PoVs still needs work.

The plan is the product. The code follows.

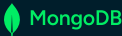


Multi-Agent Customer Service — Atlas como plano de coordenação

PoV MongoDB Atlas · 8 agentes LangGraph-style · FastAPI + React · guardrail semântico e cache em MongoDB

backend · 8631 · frontend · 5191

docs/architecture.md · ADR-001



ESTRUTURA DO REPOSITÓRIO

```
MultiAgent/
├── CLAUDE.md      memória do projeto
├── README.md      5 passos + prints
├── Implementation_plan.md
├── backend/
│   ├── app/
│   │   ├── main.py          FastAPI, rotas
│   │   ├── orchestration.py hops/fanout
│   │   ├── router.py        cheap_route
│   │   ├── agents.py        personas+tools
│   │   ├── guardrails.py
│   │   ├── retrieval.py     vector-hybrid
│   │   ├── memory.py        cascade.py
│   │   ├── database.py      DataStore
│   │   ├── security.py      policies.py
│   │   ├── budget.py        rate_limit.py
│   │   ├── metrics.py       llama.py
│   │   └── models.py        config.py
│   ├── tests/
│   │   ├── test_router.py
│   │   ├── test_policies.py
│   │   ├── test_retrieval.py
│   │   ├── test_limits.py
│   │   ├── test_cache_isolation.py
│   │   ├── test_security_config.py
│   │   └── smoke.py         caixa-preta
│   ├── run.py seed.py eval.py
│   ├── warmup.py
│   └── calibrate_thresholds.py
├── frontend/src/
│   ├── App.jsx      DEMOS_BY_IDENTITY
│   ├── api.js       REST + SSE
│   └── components/
│       ├── Timeline.jsx
│       ├── Inspector.jsx
│       └── AIBrainPanel.jsx
├── docs/
│   ├── architecture.md
│   ├── adr/ADR-001
│   ├── screenshots/ 1600x1000
│   └── env.example start.sh
```

1 - O QUE É

Atendimento multi-agente onde o **MongoDB Atlas é ao mesmo tempo plano de dados e plano de coordenação**: regras de roteamento, estado dos agentes, handoffs, memória, cache, decisões de guardrail e histórico de eval são documentos consultáveis como qualquer dado operacional.

Sem fila, sem workflow engine. Racional completo em `docs/adr/ADR-001`.

2 - OS 8 AGENTES (AGENT_REGISTRY)

orchestrator	classifica intenção só quando nenhuma regra casou
order_agent	status do pedido; única escrita restrita por allowlist
product_agent	recomendação via Vector Search do catálogo
support_agent	KB híbrida; abre support_tickets em escalação
billing_agent	faturas e cobrança
warranty_agent	políticas de garantia
loyalty_agent	resgate real de pontos + doc; de auditoria
logistics_agent	entrega; marca rescheduled_requested

Se o registry diz *N* agentes, *N* respondem — nada de documentos inertes para inflar contagem.

5 - COLEÇÕES EM JOGO

agent_registry	modelo, persona, tools, budget — editável em runtime
routing_rules	roteamento como dado, não código
agent_handoffs	fonte do Change Stream ao vivo
agent_traces	trilha por hop (\$jsonSchema válida)
agent_conversations	20 msgs / TTL 24h, retomável
memory / cache	cascata de memória; semantic_cache TTL 60min por mensagem normalizada
guardrail+ catalog / kb	denylist, policies, candidates + calibração products_catalog Vector Search; kb_articles BM25+vector RRF
negócio	orders · invoices · shipments · customers · loyalty_accounts · warranty_policies
auditoria	support_tickets · redemptions · admin_audit · eval_runs

3 - ANATOMIA DE UM TURNO

- Guardrail de entrada** — 3 camadas, mais barata primeiro
- Roteamento determinístico** `cheap_route` sobre `routing_rules`; desempate por priority
- LLM só se nenhuma regra casou** — nunca sobreescreve decisão confiante
- Cadeia até MAX_HOPS = 4** agentes, cada handoff persistido
- Ou fan-out paralelo** (order + billing) via `asyncio.gather`
- Síntese LLM** sobre o documento já buscado — retrieval 100% determinístico

4 - GUARDRAIL AUTO-REFORÇADO

- denylist estática** — substring, frase exata, custo zero
- denylist semântica** — Atlas Vector Search `denylist_autoembed_v1`, `autoEmbed` voyage-4, filtro `area+layer`; bloqueia paráfrase sem chamar LLM
- classificador LLM** — só se nada casou e a rota não estava confiante; grava o bloqueio de volta na `denylist`
- DÚVIDA** — abstenção vai para `guardrail_candidates`, revisão humana em vez de bloquear cliente legítimo

Invariantes: `vector_threshold` é medido por `calibrate_thresholds.py` contra sondas de paráfrase — nunca chutado.

8 - SEGURANÇA

- `customer_key` vem só do JWT, nunca do payload; todo filtro reconstruído no servidor
- Admin (toggle de agente, `evals` / `metrics`) atrás de X-Admin-Key; orçamento de tokens + rate limit de janela deslizante
- Fora de development o startup falha fechado: auth obrigatória, segredos com tamanho mínimo

6 - RECUPERAÇÃO

Catálogo — Vector Search com Automated Embedding (voyage-4), agregação 0.55/0.30/0.15. KB híbrida BM25+vector com RRF. Query montada em Python, nunca pelo modelo.

7 - OBSERVABILIDADE

GET `/api/events/stream` (SSE) segue Change Stream em `agent_handoffs`, filtrado ao próprio caller. Painel "coleções em ação" por turno + contadores.

12 - ARMADILHAS JÁ PAGAS

- Palavra-chave de agente posterior pode vencer a do anterior no desempate e pular o hop — reformule o prompt
- Entradas lexicais ficam fora do índice vetorial: fragmento curto fica mais perto de pergunta legítima que de ataque real
- Handoff pós-escrita roda mesmo sem mudança de estado no turno — depender disso fazia o hop sumir em replays
- Cada agente ignora em silêncio o que está fora do seu domínio; sem isso, alucina política alheia

13 - LACUNAS CONHECIDAS

Métricas in-process zeram no restart. Sem Docker/deploy — dev local via `venv+npm`. Health nunca devolve exceção crua do driver.

9 - SETUP & 10 - TESTES

```
cp .env.example .env + venv + pip install -r backend/requirements.txt + seed.py + run.py :8631 + npm run dev :5191
pytest -q · smoke.py · eval.py · warmup.py · ruff check
```

11 - API

POST `/api/auth/token` · `/api/chat` · GET `/api/agents` · `/handoffs` · `/memory/{key}` · `/events/stream` · `/metrics` · `/eval/runs` · PATCH `/api/admin/agents/{key}`